

Week 3: Sigmoid Neurons & Gradient Descent

Deep Learning – IITM BS
Notes with Intuition + Diagrams

Contents

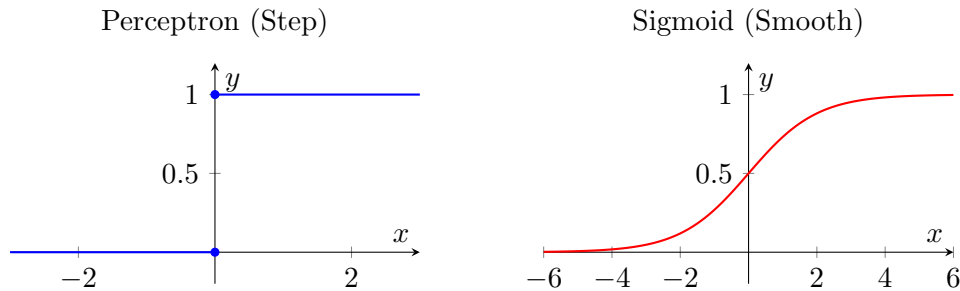
1	Why Move Beyond the Perceptron?	2
2	The Sigmoid Neuron	2
2.1	Supervised Learning Setup	2
3	Gradient Descent: The Core Idea	3
3.1	Deriving the Update Rule via Taylor Series	3
3.2	Gradient Descent Algorithm (Pseudocode)	4
4	Computing the Gradient for a Sigmoid Neuron	4
5	Sigmoid Saturation: A Critical Problem	4
6	Practice Problems	6
7	Solutions	7

1 Why Move Beyond the Perceptron?

Intuition:

The harsh step function problem The perceptron's step function is like a light switch: it's either fully OFF or fully ON, with an instant jump at the threshold. This creates two big problems:

1. Real-world decisions are rarely so binary—we want a smooth confidence score, not just yes/no
2. **Critically for learning:** the step function has a derivative of 0 almost everywhere (and undefined at the jump). You cannot do gradient-based learning on a flat, non-differentiable function!



2 The Sigmoid Neuron

Sigmoid Function

$$y = \sigma(x) = \frac{1}{1 + e^{-(w_0 + \sum_{i=1}^n w_i x_i)}}$$

- Output is always in $(0, 1)$ – can be interpreted as a **probability**
- Smooth, continuous, and **differentiable everywhere**
- As $wx + b \rightarrow +\infty$, $\sigma \rightarrow 1$; as $wx + b \rightarrow -\infty$, $\sigma \rightarrow 0$

Intuition:

Why is this the "soft" version of a perceptron? Notice: as the weight w becomes very large, the sigmoid curve becomes steeper and steeper, eventually approaching the sharp step function of the perceptron! So the perceptron is really just an extreme (infinite-weight) special case of the sigmoid neuron. The sigmoid gives us a "dial" to control how sharp or gradual the transition is.

2.1 Supervised Learning Setup

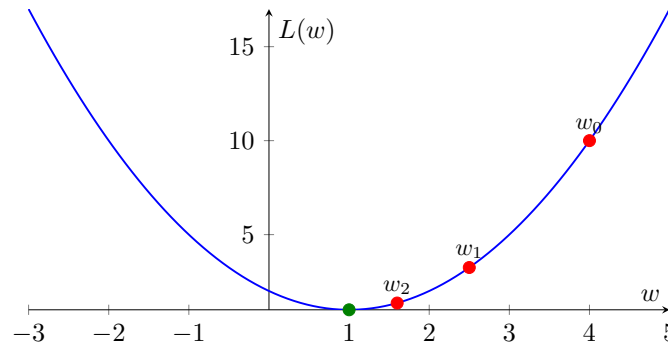
The General ML Recipe

1. **Data:** $\{x_i, y_i\}_{i=1}^N$
2. **Model:** a function $\hat{y} = \hat{f}(x; \theta)$ (e.g., $\hat{y} = \sigma(w^T x)$)
3. **Loss/Objective:** measures how wrong the model is, e.g., $L(\theta) = \sum_i (\hat{y}_i - y_i)^2$
4. **Learning algorithm:** finds θ that minimizes the loss (e.g., Gradient Descent)

3 Gradient Descent: The Core Idea

Intuition:

The blindfolded hiker analogy Imagine you're on a foggy mountain (the loss surface) and want to reach the lowest valley (minimum loss), but you can't see anything—you can only feel the slope of the ground right under your feet. The sensible strategy: always take a step in the **steepest downhill direction**. That's exactly gradient descent: the gradient tells you the direction of steepest *ascent*, so you go in the *opposite* direction.



3.1 Deriving the Update Rule via Taylor Series

Intuition:

Why Taylor series? We want to know: if I nudge θ a tiny bit in direction u , does the loss go up or down? Taylor series lets us *approximate* the new loss using only the **gradient** at the current point—without knowing the whole complicated loss landscape.

Taylor Series Derivation

$$L(\theta + \eta u) = L(\theta) + \eta u^T \nabla_{\theta} L(\theta) + \frac{\eta^2}{2!} u^T \nabla_{\theta}^2 L(\theta) u + \dots$$

For small η , higher-order terms ($\eta^2, \eta^3, \dots$) vanish:

$$L(\theta + \eta u) \approx L(\theta) + \eta u^T \nabla_{\theta} L(\theta)$$

We want the new loss to be **smaller**: $L(\theta + \eta u) - L(\theta) < 0 \Rightarrow u^T \nabla_{\theta} L(\theta) < 0$.

Let β = angle between u and $\nabla_{\theta} L(\theta)$. Since $u^T \nabla_{\theta} L(\theta) = \|u\| \|\nabla_{\theta} L(\theta)\| \cos \beta$, this quantity is **most negative** when $\cos \beta = -1$, i.e., $\beta = 180$ — meaning u points **exactly opposite** to the gradient!

The Gradient Descent Update Rule

$$w_{t+1} = w_t - \eta \nabla w_t$$

$$b_{t+1} = b_t - \eta \nabla b_t$$

η = learning rate (step size). The minus sign is exactly because we move *opposite* the gradient (steepest ascent direction) to *descend*.

3.2 Gradient Descent Algorithm (Pseudocode)

Algorithm

1. $t \leftarrow 0$, initialize w, b randomly
2. **While** $t < \text{max_iterations}$:
 - $w_{t+1} \leftarrow w_t - \eta \nabla w_t$
 - $b_{t+1} \leftarrow b_t - \eta \nabla b_t$
 - $t \leftarrow t + 1$

4 Computing the Gradient for a Sigmoid Neuron

For a single sigmoid neuron $f(x) = \sigma(wx + b)$, with squared error loss:

Sigmoid Gradients – Derive from Chain Rule

$$\nabla w = \frac{\partial L(x)}{\partial w} = (f(x) - y) \cdot f(x) \cdot (1 - f(x)) \cdot x$$

$$\nabla b = \frac{\partial L(x)}{\partial b} = (f(x) - y) \cdot f(x) \cdot (1 - f(x))$$

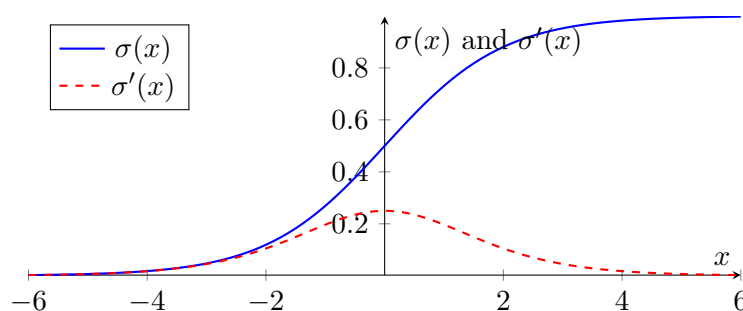
Intuition:

Reading this formula intuitively This is three multiplied factors, each with meaning:

- $(f(x) - y)$: **how wrong** the prediction is (error signal)
- $f(x)(1 - f(x)) = \sigma'(x)$: **how sensitive** the sigmoid is right now (this is the sigmoid's own derivative—maximal at 0.5, near-zero when saturated close to 0 or 1)
- x : **how much this input contributed** (chain rule: derivative of $w x + b$ w.r.t. w is x)

This “error $\times$ local sensitivity $\times$ input contribution” pattern repeats *everywhere* in backpropagation (Week 4)!

5 Sigmoid Saturation: A Critical Problem



The Saturation Trap

When x is very large (positive or negative), $\sigma(x)$ flattens out near 0 or 1, and $\sigma'(x) \rightarrow 0$. Since $\nabla w \propto \sigma'(x)$, the gradient **vanishes**, and learning **stalls**—the neuron is “stuck” and stops updating meaningfully. This is called **saturation** and is the seed of the vanishing gradient problem you’ll see amplified in deep networks and RNNs.

Key Values to Memorize

$$\begin{aligned}\sigma(0) &= 0.5, & \sigma'(0) &= 0.25 \text{ (maximum slope)} \\ \sigma(1) &\approx 0.731, & \sigma(-1) &\approx 0.269, & \sigma(2) &\approx 0.881, & \sigma(-2) &\approx 0.119\end{aligned}$$

6 Practice Problems

Problem 1: Sigmoid Evaluation [2 marks]

For a sigmoid neuron with $w = 2$, $b = -1$, compute the output for $x = 0.5$.

Problem 2: Gradient Computation [3 marks]

Sigmoid neuron: $w = 1$, $b = 0$, single data point $x = 0$, $y = 1$.

- (a) Compute $f(x) = \sigma(wx + b)$.
- (b) Compute ∇w and ∇b using the formulas above.

Problem 3: One GD Step [3 marks]

Using the gradients from Problem 2, and learning rate $\eta = 0.5$, compute the updated w_1 and b_1 after one gradient descent step (starting from $w_0 = 1$, $b_0 = 0$).

Problem 4: Saturation Identification [2 marks, MCQ]

A sigmoid neuron has $wx + b = 8$. What can you say about the gradient ∇w for this data point (assuming a typical, non-zero error)?

- A. Very large B. Very close to zero (saturated) C. Exactly 0.25 D. Negative

Problem 5: Taylor Series Direction [2 marks]

If $\nabla_{\theta} L(\theta) = [3, -4]^T$, what is the unit vector u that gives the steepest **decrease** in loss?

Problem 6: Multiple GD Steps [3 marks]

$L(w) = (w - 3)^2$. Starting at $w_0 = 7$, learning rate $\eta = 0.5$. Compute w_1 and w_2 .
(Hint: $\nabla L(w) = 2(w - 3)$.)

7 Solutions

Problem 1: $wx + b = 2(0.5) - 1 = 0$. $\sigma(0) = \mathbf{0.5}$.

Problem 2: (a) $wx + b = 1(0) + 0 = 0$. $f(x) = \sigma(0) = 0.5$.

(b) $\nabla w = (0.5-1)(0.5)(0.5)(0) = 0$ (since $x = 0$). $\nabla b = (0.5-1)(0.5)(0.5) = -0.5 \times 0.25 = -\mathbf{0.125}$.
 $\nabla w = \mathbf{0}$.

Problem 3: $w_1 = w_0 - \eta \nabla w = 1 - 0.5(0) = \mathbf{1}$. $b_1 = b_0 - \eta \nabla b = 0 - 0.5(-0.125) = \mathbf{0.0625}$.

Problem 4: B. At $wx + b = 8$, $\sigma(8) \approx 0.9997$ (near 1), so $\sigma'(8) \approx 0.0003$, very close to zero—the neuron is saturated.

Problem 5: Steepest decrease direction is $-\nabla L / \|\nabla L\|$. $\|\nabla L\| = \sqrt{9+16} = 5$. $u = -[3, -4]/5 = [-\mathbf{0.6}, \mathbf{0.8}]$.

Problem 6: $\nabla L(7) = 2(7-3) = 8$. $w_1 = 7 - 0.5(8) = 7 - 4 = 3$? Let's verify: $w_1 = 7 - 0.5 \times 8 = 7 - 4 = \mathbf{3}$.

$\nabla L(3) = 2(3-3) = 0$. $w_2 = 3 - 0.5(0) = \mathbf{3}$ (already at the minimum!).